CUDA Ray Tracing

Ever wondered how games make reflections and shadows look so real? That's ray tracing. In this project you'll build your own from scratch and find out just how fast you can render one.

Overview

Ray tracing produces realistic images by simulating how light rays travel and bounce through a scene. It is conceptually simple and naturally parallel — every pixel is computed independently. At the same time, its recursive nature and irregular control flow make it an interesting GPU workload.

In this project you will build a working ray tracer in CUDA, add reflections and shadows, then make it faster. The emphasis of this course is **not** on producing pretty pictures. By the end of the project, you should be able to:

- Map a 2D image onto a CUDA grid of threads.
- Convert a naturally recursive algorithm into a GPU-friendly implementation.
- Measure GPU performance and reason about warp divergence, occupancy, and memory behaviour.

Starting Point

You are given a small CPU ray tracer (starter_raytracer_cpu.cpp) that renders 3 spheres on a checkerboard floor with diffuse shading and hard shadows, to give you some graphics knowledge to begin with.

Your first deliverable is to understand the code and port it to CUDA, producing the same image as the CPU implementation. You should also extend it with a single reflection bounce (both the sphere(s) and the checkerboard floor should be reflective).

Multi-Bounce Reflections

You can later extend to multiple reflection bounces and study their effect on visual quality, execution time, and GPU utilization, measured with CUDA events and presented as a function of reflection depth.

Open-Ended Extensions

You may pursue additional extensions of your choice to encourage creativity and exploration, or to make a more computationally intensive workload. Examples: additional spheres or geometric shapes, custom scenes, textures, bounding volume hierarchies (BVH), and higher resolutions.

Optimizations may target GPU efficiency (memory access, occupancy, reducing divergence, launch-configuration tuning) and/or algorithmic improvements (e.g. acceleration structures like a BVH).

These are ***optional*** — you can still continue with just spheres and a plane. Scene complexity / artistic quality is not part of the grading criteria.

Deliverables

1. CUDA implementation that builds and runs on a GPU.
2. Rendered images — baseline, multi-bounce reflections, and any optional extensions.
3. Technical report. It must include:
    - a CPU vs. CUDA comparison and your CUDA mapping strategy (how pixels map to threads/blocks);
    - scaling with image resolution and/or bounce depth, with performance measurements;
    - the impact of at least one optimization — including optimization attempts that did not help and the likely reasons why;
    - explicit reasoning about warp divergence, occupancy, register pressure, and memory behaviour.

Notes

- Time the kernels with CUDA events (warm up, then synchronize) — not wall-clock around the whole program. A trivial scene is dominated by launch/transfer overhead, so scale up resolution and bounce depth until timings are meaningful before optimizing.
- Suggested stopping conditions for multi-bounce reflection: maximum bounce depth reached, ray leaves the scene, or ray energy falls below a threshold and its contribution becomes negligible
- Reflection math: reflected direction `r = d − 2 (d·n) n` for incoming direction `d` and unit normal `n`.
- Link to Github repo hosting the problem statement: w-vaishnavi/raytracing
- *On using LLMs* - The joy of this project is to build a mini version yourself. Polished ray tracers exist and an LLM can hand you one in seconds - the point is not to compete with them. We suggest limiting LLM use to explaining snippets, clear doubts or discuss ideas.